

# Hardware Acceleration of CNN for MNIST Digit Recognition Using Systolic Arrays

Prayanshu Sharma, Vedant Saxena, Anirudh Chaturvedi

Department of Electronic Systems Engineering

Indian Institute of Science, Bengaluru, India

prayanshus@iisc.ac.in, vedantsaxena@iisc.ac.in, anirudhc@iisc.ac.in

**Abstract**—Convolutional Neural Networks (CNNs) achieve high accuracy in handwritten digit recognition on the MNIST dataset [1] but incur significant latency and energy overhead on general-purpose processors due to compute-intensive convolution and fully connected operations. This project presents a dedicated systolic-array-based hardware accelerator for efficient CNN inference. The architecture integrates a two-dimensional processing element array for parallel multiply-accumulate operations, on-chip buffering for efficient data reuse, and centralized control for scheduling. Post-processing units for activation, pooling, and classification are included to enable end-to-end inference, and fixed-point quantization is employed to improve area and energy efficiency. The proposed accelerator is evaluated on MNIST in terms of throughput, latency, resource utilization, and classification accuracy, demonstrating the effectiveness of systolic arrays for low-latency and energy-efficient CNN inference.

**Index Terms**—CNN acceleration, Systolic Array, MNIST, Hardware Accelerator, FPGA/ASIC, Deep Learning Inference

## I. INTRODUCTION AND MOTIVATION

Handwritten digit recognition is a fundamental problem in computer vision with applications in optical character recognition, automated document processing, and embedded intelligence systems. Convolutional Neural Networks (CNNs) have become the dominant approach for such tasks due to their strong representation capability and high classification accuracy on benchmark datasets such as MNIST. However, CNN inference is computationally intensive, as it is dominated by convolution and fully connected layers that require a large number of multiply-accumulate (MAC) operations. When executed on general-purpose processors, these workloads incur high latency and energy consumption, limiting their applicability in real-time and resource-constrained environments such as edge devices and embedded systems.

Recent advances in domain-specific hardware accelerators, such as the Tensor Processing Unit (TPU) and energy-efficient CNN accelerators, demonstrate that specialized architectures can significantly improve performance and energy efficiency for deep learning workloads [2]–[4]. Among these, systolic arrays provide an attractive solution due to their regular structure, high degree of parallelism, and efficient data reuse through local inter-PE communication [5]. By streaming inputs and weights through a two-dimensional grid of processing elements, systolic arrays minimize off-chip memory accesses and enable high-throughput matrix multiplication, which directly maps to convolution and fully connected operations in CNNs.

This project is motivated by the need to design a compact and efficient CNN inference accelerator using systolic array architecture for MNIST digit recognition, targeting improved throughput, reduced latency, and lower energy consumption compared to conventional processor-based implementations.

## II. BACKGROUND STUDY

Convolutional Neural Networks (CNNs) achieve high accuracy for handwritten digit recognition on the MNIST dataset by learning hierarchical features through convolution, activation, pooling, and fully connected layers [6], [7]. However, CNN inference is dominated by convolution and matrix multiplication operations involving a large number of multiply-accumulate (MAC) computations, leading to high latency and energy consumption on general-purpose processors. As model complexity increases, the computational and memory access demands further limit software-based implementations for real-time and embedded deployments.

Systolic arrays [8] provide an efficient architecture for CNN acceleration by enabling regular dataflow, pipelining, and operand reuse across a grid of processing elements, thereby reducing memory bandwidth requirements. Furthermore, fixed-point quantization significantly reduces hardware complexity and power consumption with minimal impact on inference accuracy [9]. These observations motivate the proposed systolic-array-based CNN accelerator for efficient handwritten digit recognition.

## III. INNOVATIVE FEATURES AND DESIGN GOALS

The proposed accelerator architecture is designed in alignment with the block-level diagram presented in this work and targets efficient end-to-end CNN inference for MNIST digit recognition. The key innovative features and design goals are summarized as follows:

- **Systolic Array-Based Compute Engine:** A two-dimensional array of processing elements (PEs) is employed to perform parallel multiply-accumulate (MAC) operations for convolution and fully connected layers, exploiting both spatial and temporal data reuse.
- **On-Chip Global Buffering:** Dedicated global buffers are used to store input feature maps, weights, and intermediate outputs, thereby reducing off-chip memory accesses and improving data locality.

- **Centralized Control and Scheduling Unit:** A centralized controller orchestrates data movement, address generation, and tiling of inputs and weights to efficiently feed the systolic array and manage pipelined execution.
- **Partial Sum Accumulation Support:** Intermediate partial sums generated within the systolic array are accumulated and buffered to support multi-channel convolution and large matrix multiplication.
- **Integrated Post-Processing Pipeline:** Hardware modules for activation (ReLU), pooling, and final classification (argmax) are integrated to enable complete end-to-end CNN inference within the accelerator.
- **Fixed-Point Quantized Arithmetic:** The architecture supports fixed-point computation to reduce hardware complexity, memory footprint, and power consumption while maintaining acceptable classification accuracy.
- **Scalability and Reconfigurability:** The systolic array and control logic are designed to be scalable to different array dimensions and configurable to support varying kernel sizes and CNN layer parameters.

#### IV. PROPOSED ARCHITECTURE

The proposed architecture, as shown in Fig. 1, implements a hardware accelerator for Convolutional Neural Networks (CNNs) targeted at the Artix-7 XC7A200T FPGA. The overall system operation is as follows: a host PC transmits an input image to the FPGA via a serial link; the FPGA then executes the CNN inference on the provided image; and finally, the detected digit is outputted to a 7-segment display for user verification.

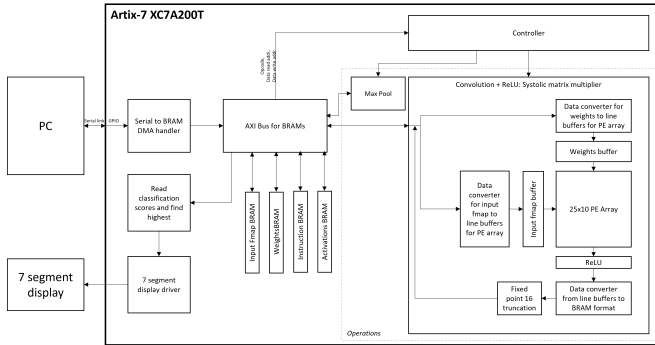


Fig. 1. Overall block diagram of the proposed CNN hardware accelerator.

The internal module hierarchy of the accelerator is structured as follows:

- **Serial to BRAM DMA handler:** Manages Direct Memory Access transfers, converting incoming serial data from the PC into a parallel format suitable for writing to the on-chip Block RAMs.
- **AXI Bus for BRAMs:** Used for routing data between the BRAMs and all other modules.
- **Block RAMs:** On-chip memory elements used for fast, localized data storage.

- **Input Fmap BRAM:** Stores the initial feature map for each layer. Each layer's input is memory mapped here.
- **WeightsBRAM:** Holds the pre-trained weights and biases required for the CNN computations.
- **Instruction BRAM:** Stores the opcodes and memory addresses (read/write) that dictate the control flow of the accelerator.
- **Activations BRAM:** Stores the activation outputs generated by each layer. Each layer's output is memory mapped here.
- **Controller:** Reads instructions from the Instruction BRAM to coordinate data routing and govern the operation of the functional units.
- **Operations:** The logical grouping of the main mathematical processing blocks connected to the AXI bus.
  - **Max Pool:** An independent module that performs max pooling operations to downsample the spatial dimensions of the feature maps, reading and writing intermediate data via the AXI bus.
  - **Convolution + ReLU: Systolic matrix multiplier:** The core computational engine responsible for parallelizing the heavy multiply-accumulate (MAC) operations of the CNN.
    - \* **Data converter for weights to line buffers for PE array:** Retrieves weight data via the AXI bus and formats it into sequential line buffers for structured injection into the PE array.
    - \* **Weights buffer:** Temporarily caches formatted weights to ensure a continuous pipeline of data to the compute elements.
    - \* **Data converter for input fmap to line buffers for PE array:** Formats the incoming feature map data from BRAM into sequential line buffers.
    - \* **Input fmap buffer:** Caches the formatted feature maps to keep the PE array adequately fed with input data.
    - \* **PE Array:** A 25x10 (tentative) grid of Processing Elements that perform the highly parallelized MAC operations required for convolutions and fully connected layers. The size of the grid is chosen to optimally accommodate LeNet-5 CNN model.
    - \* **ReLU:** Hardware block that applies the Rectified Linear Unit non-linear activation function to the outputs of the PE array.
    - \* **Data converter from line buffers to BRAM format:** Repackages the processed and activated data back into the standard format required for storage in the BRAM.
    - \* **Fixed point 16 truncation:** Converts 32-bit accumulator results back to a 16-bit fixed-point format to maintain consistent data widths across the hardware before routing back to the AXI bus.
- **Read classification scores:** Evaluates the final output

array to find the index of the maximum value, which corresponds to the network's predicted digit (0-9).

- **7 segment display driver:** Translates the final predicted digit into the specific GPIO signals needed to illuminate the corresponding numbers on the FPGA board's physical display.

## V. ESTIMATED TIMELINE

The plan and milestones for the project are summarized in Table I

TABLE I  
PROJECT TIMELINE (7 WEEKS)

| Phase | Task Description                                                                                                                                                                                                                                                                                                                                                                                     | Duration  |
|-------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------|
| P1    | <ul style="list-style-type: none"> <li>Finalization of CNN model</li> <li>Memory map</li> <li>Tentative architecture</li> <li>Convolution to Systolic array dataflow algorithm: Decide between im2col approach or running transformation</li> </ul>                                                                                                                                                  | Week 1    |
| P2    | <ul style="list-style-type: none"> <li>Implementation of Global buffer memories: <ul style="list-style-type: none"> <li>AXI Bus</li> <li>Memory banks as per memory map table</li> </ul> </li> </ul>                                                                                                                                                                                                 | Week 2    |
| P3    | <ul style="list-style-type: none"> <li>RTL implementation of systolic matrix multiplier: <ul style="list-style-type: none"> <li>PE block</li> <li>Parameterized PE array (M×N)</li> <li>Weight, input and output buffers</li> <li>Data sequencer for this multiplier</li> </ul> </li> <li>Verification of this multiplier</li> </ul>                                                                 | Weeks 2–3 |
| P4    | <ul style="list-style-type: none"> <li>Implementation of convolution operator (Wrapper logic around systolic multiplier for convolution): <ul style="list-style-type: none"> <li>Conversion of weights and input fmap to systolic array input format</li> <li>ReLU unit</li> <li>Conversion of multiplier output back to conv output shape</li> <li>Fixed point 16 truncation</li> </ul> </li> </ul> | Week 4    |
| P4    | <ul style="list-style-type: none"> <li>Implementation of Controller FSM: <ul style="list-style-type: none"> <li>Conversion of CNN ops to Op-codes and addresses</li> <li>FSM Design</li> <li>FSM RTL Coding</li> <li>Verification of controller</li> </ul> </li> </ul>                                                                                                                               | Week 4    |
| P5    | <ul style="list-style-type: none"> <li>Top level integration <ul style="list-style-type: none"> <li>Simulation, validation</li> <li>FPGA synthesis, timing closure</li> <li>On-board implementation (Artix-7 XC7A200T)</li> </ul> </li> </ul>                                                                                                                                                        | Week 5    |
| P6    | <ul style="list-style-type: none"> <li>Performance evaluation (latency, throughput, resource utilization)</li> <li>Documentation and presentation preparation</li> </ul>                                                                                                                                                                                                                             | Week 6    |

## REFERENCES

- [1] Y. LeCun and C. Cortes. MNIST handwritten digit database, 1998. [Online]. Available: <http://yann.lecun.com/exdb/mnist/>.
- [2] N. P. Jouppi et al. In-datacenter performance analysis of a Tensor Processing Unit. In *Proc. 44th Annual International Symposium on Computer Architecture (ISCA)*, pages 1–12, 2017.
- [3] Y. Chen et al. Eyeriss: An energy-efficient reconfigurable accelerator for deep convolutional neural networks. *IEEE Journal of Solid-State Circuits*, 52(1):127–138, 2016.
- [4] C. Zhang et al. Optimizing FPGA-based accelerator design for deep convolutional neural networks. In *Proc. ACM/SIGDA International Symposium on Field-Programmable Gate Arrays (FPGA)*, pages 161–170, 2015.
- [5] V. Sze, Y.-H. Chen, T.-J. Yang, and J. S. Emer. Efficient processing of deep neural networks: A tutorial and survey. *Proceedings of the IEEE*, 105(12):2295–2329, 2017.
- [6] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998.
- [7] A. Krizhevsky, I. Sutskever, and G. Hinton. ImageNet classification with deep convolutional neural networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2012.
- [8] H. T. Kung and C. E. Leiserson. Systolic arrays for VLSI. In *Sparse Matrix Proceedings*, pages 256–282, 1982.
- [9] S. Han, H. Mao, and W. J. Dally. Deep compression: Compressing deep neural networks with pruning, trained quantization and Huffman coding. In *International Conference on Learning Representations (ICLR)*, 2016.